Contextual Semantic Tokenization: A Linguistically-Grounded Alternative to Subword Segmentation for Language Modeling

Emad Jumaah

Abstract

We introduce Contextual Semantic Tokenization (CST), a tokenization method that segments text into linguistically-motivated semantic units rather than statistically-derived subword fragments. The approach originates from an observation about Arabic triconsonantal morphology: Arabic roots and patterns constitute a natural algebraic system in which root × pattern = concept, providing semantic structure directly in the vocabulary. We generalize this insight to a universal framework applicable to any language. CST maps words to a closed set of semantic fields through lemmatization, morphological decomposition, and field resolution, producing structured tokens such as `CMP:write:agent` for "writer" or `REL:causes` for "because." In controlled experiments training GPT-2 language models from scratch on 100K English sentences, we compare CST against SentencePiece BPE at matched vocabulary sizes (8K and 32K) and identical model architectures, ensuring equal parameter counts. Using bits-per-character (BPC) as the evaluation metric, CST achieves 1.13 BPC versus 1.75 BPC for SentencePiece at 8K vocabulary (35.5% reduction) and 1.23 versus 1.65 at 32K vocabulary (25.2% reduction). CST also produces shorter token sequences (22.1 tokens/sentence versus 31.7 for BPE-8K), yielding proportionally faster training. These results suggest that encoding linguistic structure directly into the tokenization layer provides a stronger inductive bias for language modeling than frequency-based subword segmentation — a principle first made visible by Arabic morphology and here generalized to English and beyond.

1. Introduction

Tokenization is the first transformation applied to text before it reaches a neural language model, yet it remains largely disconnected from linguistic theory. The dominant approach, Byte Pair Encoding (BPE; Sennrich et al., 2016) and its variants (WordPiece, Unigram), constructs a vocabulary by iteratively merging frequent character sequences. The resulting tokens are statistical artifacts: they reflect surface-level co-occurrence patterns in the training corpus rather than semantic or grammatical units. A word like "unhappiness" might be split into "un," "happ," "iness" — fragments that carry no consistent linguistic meaning across contexts.

The motivation for this work comes from an unexpected source: Arabic morphology.

Arabic is a Semitic language built on a triconsonantal root system. Every Arabic word is derived from a three-consonant root (الجذر, *al-jidhr*) through the application of a vowel pattern (الوزن, *al-wazn*). The root ك-ت-ب (k-t-b) encodes the semantic domain of writing. Applying ten canonical patterns produces ten distinct words:

Pattern (وزن)	Operator	Result	Meaning
فَاعِل	agent	كاتب	writer
مَفْعُول	patient	مكتوب	letter / written thing
مَفْعَلَة	place	مكتبة	library
فِعَال	instance	كتاب	a book
فُعُول	plural	كُتُب	books
اِسْتِفْعَال	seek	استكتاب	requesting to write
تَفَاعُل	mutual	تكاتُب	exchanging letters
مُفَاعَلَة	process	مُكاتَبة	correspondence
فَعَّال	intensifier	كتَّاب	professional scribe
مُفْعِل	causer	مُكتِب	one who dictates

This is not merely linguistic description — it is an algebra. Root × Pattern = Concept. The semantic field (writing) and the morphological role (agent, place, patient) compose deterministically into a specific meaning. Crucially, this structure is encoded directly in the word's surface form: a morphological analyzer can recover both root and pattern from any Arabic word without external context.

This algebraic property of Arabic morphology suggests a hypothesis: **what if tokens in a language model's vocabulary carried the same structured information?** Instead of arbitrary subword fragments, what if each token explicitly encoded its semantic field and its morphological role?

We generalize this insight beyond Arabic into a universal framework. All human languages encode meaning compositionally in word structure — through suffixes and prefixes in English, through agglutination in Turkish, through noun classes in Swahili, through the same Semitic root system in Hebrew and Arabic. These are different surface realizations of a shared principle: **morphology encodes semantic relationships**. CST extracts those relationships and makes them explicit in the token vocabulary.

The Arabic root-pattern system makes this principle maximally visible because the composition is surface-transparent: the root and pattern can be read directly from the word's consonant skeleton and vowel template. English requires a lookup step (lemmatization + affix detection) to recover the same information. But the information is there in both languages — Arabic simply makes it harder to miss.

This paper makes the following contributions:

1. We trace the conceptual origin of CST to Arabic triconsonantal morphology, showing how root × pattern composition generalizes to a universal tokenization principle.
2. We describe the CST pipeline, a seven-stage tokenizer that produces typed, linguistically-annotated tokens for English.
3. We present a controlled comparison against SentencePiece BPE at matched vocabulary sizes and parameter counts, isolating tokenization strategy as the sole variable.
4. We report a 35.5% BPC reduction at 8K vocabulary and 25.2% at 32K vocabulary, demonstrating that linguistic structure in tokenization improves language modeling efficiency.
5. We analyze CST's cross-lingual properties, showing that its semantic token space is language-agnostic and that Arabic and other morphologically rich languages are natural beneficiaries.

---

2. Related Work

Subword tokenization. BPE (Sennrich et al., 2016) and its variants — WordPiece (Schuster & Nakajima, 2012), Unigram (Kudo, 2018), and SentencePiece (Kudo & Richardson, 2018) — remain the standard tokenization methods for neural language models. These methods optimize compression of the training corpus without reference to linguistic structure. Recent work has noted limitations: Bostrom & Durrett (2020) show that BPE segmentation often misaligns with morphological boundaries, and Rust et al. (2021) demonstrate that tokenizer quality degrades for under-represented languages in multilingual models.

Morphological tokenization. Attempts to incorporate morphology into tokenization include Morfessor (Creutz & Lagus, 2007), which segments words into morphs using minimum description length, and the work of Ataman & Federico (2018), who apply supervised morphological segmentation to neural machine translation. These approaches improve handling of morphologically rich languages but do not assign semantic content to tokens. Arabic-specific work includes CAMeLBERT (Inoue et al., 2021) and AraBERT (Antoun et al., 2020), which apply morphological preprocessing before BERT-style training but retain subword vocabularies at the model input layer.

Semitic morphology as computation. The algebraic structure of Semitic root-pattern morphology has been studied formally since McCarthy (1981), who introduced autosegmental phonology to model the nonconcatenative nature of Arabic and Hebrew word formation. Beesley & Karttunen (2003) developed finite-state transducer models for Semitic morphology, and Kiraz (2001) extended this to multitiered representations. These works treat morphology as a parsing problem — recovering roots and patterns from surface forms. CST inverts this: it uses the recovered structure as a generative vocabulary for neural models.

Semantic representations in NLP. Semantic field theory (Trier, 1931) and frame semantics (Fillmore, 1982) propose that word meanings organize into structured fields. FrameNet (Baker et al., 1998) and WordNet (Miller, 1995) provide computational inventories of semantic relations. However, these resources have not been applied at the tokenization layer — they are typically used downstream for tasks such as semantic role labeling.

CST's position. CST bridges the gap between morphological tokenization and semantic representation. It draws on Arabic morphology's algebraic structure as its conceptual foundation, extends the root-pattern decomposition to English through lemmatization and affix detection, and encodes the result directly in the token vocabulary that the language model trains on. Unlike morphological segmenters, CST maps tokens to semantic fields. Unlike semantic databases, CST operates at the tokenization layer. The key novelty is that the semantic structure is not a downstream annotation — it is the vocabulary itself.

---

3. From Arabic Algebra to Universal Tokenization

Before describing the CST pipeline, we make explicit the conceptual path from Arabic morphology to the general framework. This origin is not incidental — the design decisions in CST reflect the structure of Arabic morphology directly.

3.1 Arabic as the Blueprint

In Arabic, every content word is the product of two orthogonal components:

- **Root (الجذر):** A set of three (occasionally four) consonants encoding a semantic domain. The root ك-ت-ب covers the entire domain of writing and recording; the root ع-ل-م covers the domain of knowledge and learning; the root ج-م-ع covers gathering and assembly.

- **Pattern (الوزن):** A morphological template that specifies the word's grammatical role. The pattern فَاعِل (fā'il) produces the active agent of any root; مَفْعَلَة (maf'ala) produces the place associated with any root; مَفْعُول (maf'ūl) produces the passive patient of any root.

The composition root × pattern is deterministic and productive: given any root and any pattern, the resulting word's meaning is predictable. This is the algebra. كتب × فاعل = كاتب (writer); علم × مفعلة = مدرسة (school); جمع × استفعال = اجتماع (assembly/meeting).

CST's token type `CMP:field:role` is a direct encoding of this algebra. `CMP:write:agent` is the computational representation of كتب × فاعل. `CMP:know:place` is the computational representation of علم × مفعلة. The Arabic morphological system, studied and refined over fourteen centuries of linguistic scholarship, provides the conceptual foundation. CST makes it computable and language-independent.

3.2 Generalization to English

English does not have an Arabic-style root system, but it has morphological structure that encodes the same relationships less transparently:

Arabic (explicit)	English (recoverable)	CST token
كاتب (root:كتب + pattern:فاعل)	writer (lemma:write + suffix:-er)	`CMP:write:agent`
مكتبة (root:كتب + pattern:مفعلة)	library (semantic field:write + role:place)	`CMP:write:place`
مكتوب (root:كتب + pattern:مفعول)	document (semantic field:write + role:instance)	`CMP:write:instance`
معلم (root:علم + pattern:مفعل)	teacher (lemma:teach + suffix:-er + semantic causer)	`CMP:know:causer`

In Arabic, the extraction is morphological: root and pattern are read from the surface form. In English, the extraction requires lemmatization (write ← writer), affix detection (-er → agent), and semantic field lookup (write → field:write). The output is the same structured token. The Arabic route is more direct; the English route requires more processing. Both arrive at the same vocabulary.

3.3 Universal Semantic Fields

The semantic fields in CST — write, know, move, create, send, think, gather — are not English categories. They are universal semantic primitives that appear in all human languages because they describe fundamental human activities. McCarthy (1981) and subsequent work in linguistic typology have documented that semantic domains such as motion, cognition, creation, and communication are universal across unrelated language families.

This universality is the key to CST's cross-lingual potential. When Arabic الباحث (researcher) and English "researcher" both map to `CMP:science:agent`, a model training on both languages in the same token space learns a unified representation of the concept. The LIT tokens — function words, proper nouns, unresolved surface forms — carry language-specific information. The semantic tokens carry universal information. A multilingual model built on CST shares its semantic core entirely across languages; only the LIT layer varies.

We note, as an observation rather than a design claim, that this property bears a structural resemblance to what constructed universal language projects — such as Esperanto or Interlingua — attempted to achieve deliberately. The distinction is one of origin: those projects imposed a shared vocabulary through explicit design. In CST, the shared semantic layer was not an intended outcome; it arises as a consequence of grounding tokens in morphological and semantic primitives that, as linguistic typology

suggests, are independently present in all human languages. Whether this emergent cross-lingual alignment translates into measurable benefits for multilingual modeling is an open empirical question that we leave to future work.

---

4. Method

4.1 Overview

CST is a seven-stage pipeline that transforms raw text into a sequence of typed tokens. Each token belongs to one of six types:

Type	Format	Example	Description
CMP	`CMP:field:role`	`CMP:write:agent`	Composed: semantic field + morphological role
ROOT	`ROOT:field`	`ROOT:move`	Semantic field, no morphological derivation
REL	`REL:relation`	`REL:causes`	Grammatical or logical relation
STR	`STR:structure`	`STR:negation`	Sentence-level structural marker
LIT	`LIT:surface`	`LIT:the`	Literal token (function words, entities, unresolved words)
SPECIAL	`[PAD]` , `[UNK]`	`[UNK]`	Padding and unknown tokens

The CMP token directly encodes the Arabic algebra: field = root semantic domain, role = pattern operator. ROOT encodes a semantic field without a recoverable morphological role. REL and STR encode sentence-level structure. LIT is the fallback for words that do not participate in the semantic algebra.

4.2 Pipeline Stages

Stage 1: Normalization. Input text is lowercased, Unicode smart quotes are replaced with ASCII equivalents, and whitespace is normalized. This ensures consistent downstream processing without losing semantic content.

Stage 2: Structure detection. A set of regular expressions scans the full sentence for structural patterns before word-level processing begins. Six structure types are detected:

- `STR:question` — terminal question mark
- `STR:negation` — negation markers (*not*, *never*, *cannot*, etc.)
- `STR:condition` — conditional markers (*if*, *unless*, *whenever*, etc.)
- `STR:future` — future tense markers (*will*, *shall*, *going to*)
- `STR:past` — past tense markers (*was*, *were*, *had*, *did*)
- `STR:emphasis` — terminal exclamation mark

Structure tokens are prepended to the token sequence, providing the model with a sentence-level frame before processing individual words. This mirrors the function of Arabic grammatical particles (هل for questions, لا for negation, سـ for future) which operate at the sentence level.

Stage 3: Word splitting. The normalized text is split on whitespace and punctuation boundaries. Contraction fragments (*'s*, *n't*, *'re*, *'ll*, *'ve*, *'d*, *'m*) and bare punctuation are filtered, as their semantic content is captured by structure detection or is negligible.

Stage 4: Named entity recognition. The NLP library compromise.js identifies named entities (people, places, organizations) in the original text. Recognized entities bypass morphological decomposition and field resolution, emitting directly as `LIT:surface` tokens to preserve proper nouns.

Stage 5: Lemmatization. Each word is lemmatized using compromise.js, which reduces verbs to infinitive form and nouns to singular. Results are cached in a persistent map to avoid redundant NLP calls; in a 100K-sentence run, approximately 78K unique word forms are encountered and cached.

Stage 6: Morphological decomposition. The surface form and lemma are compared to detect affixation:

- *Prefix detection* (9 prefixes): Prefixes such as *un-*, *re-*, *dis-*, *pre-*, *mis-*, *over-*, *co-*, *out-*, *non-* are matched at word boundaries, producing roles like `negate`, `repeat`, `before`. A minimum stem length of 3 characters prevents false positives.
- *Suffix detection* (25 suffixes): When the surface form differs from the lemma at the end, the difference is matched against known suffixes. Suffixes are matched greedily (longest first) and produce roles such as `agent` (*-er*, *-or*, *-ist*), `instance` (*-tion*, *-ment*, *-ing*), `state` (*-ness*, *-ity*), `possible` (*-able*, *-ible*), `past` (*-ed*), `plural` (*-s*, *-es*), among others.

The mapping from affix to role mirrors the Arabic pattern system: where Arabic uses فاعل to mark the agent of any verb, English uses *-er*. The roles themselves — agent, patient, place, instance, causer — are identical across the two systems, because they reflect universal semantic relationships, not language-specific conventions.

Stage 7: Token emission. Each word is resolved to a token through a priority cascade:

1. If the word is a recognized named entity → `LIT:surface`
2. If the word is a digit sequence → `ROOT:size`
3. If the word appears in the relation map (~245 entries covering prepositions, conjunctions, modals, quantifiers, adverbs) → `REL:relation`
4. If the word is a function word (~95 entries: articles, pronouns, auxiliaries) → `LIT:surface`
5. The morphological root is looked up in a semantic field dictionary (~2,400 lemma-to-field mappings covering ~45 universal fields). A silent-e recovery heuristic (e.g., *writ* → *write*) and nested suffix stripping expand coverage. If a field is found:
   - With a morphological role → `CMP:field:role` (e.g., *writer* → `CMP:write:agent`)
   - Without a role → `ROOT:field` (e.g., *write* → `ROOT:write`)
6. Fallback → `LIT:surface`

4.3 Semantic Field Inventory

The semantic field dictionary maps lemmas to one of approximately 45 universal fields. Representative examples:

Lemma cluster	Field
consider, think, reason, judge, analyze	`think`
go, travel, arrive, depart, return	`move`
make, build, create, produce, design	`create`
know, learn, understand, study, teach	`know`
send, transmit, dispatch, deliver	`send`
write, record, document, publish	`write`
see, observe, watch, notice, perceive	`see`

These fields are designed to be universal: the same fields apply across languages, since semantic primitives such as *move*, *think*, *create*, and *know* appear in all human languages. This is not an assumption — it is the finding of cross-linguistic typology (Fillmore, 1982;

Miller, 1995). The Arabic roots used in the original Arabic Algebra Engine (كتب for write, علم for know, رسل for send, جمع for gather) map directly to these fields. The field names are language-neutral labels for semantic domains that Arabic identified structurally.

4.4 Vocabulary Capping

In its unconstrained form, CST produces approximately 846 semantic tokens (CMP, ROOT, REL, STR, SPECIAL) and a variable number of LIT tokens for words not covered by the semantic field dictionary. For controlled comparison against a fixed-vocabulary baseline, we cap the CST vocabulary to a target size V by:

1. Retaining all semantic tokens (~846)
2. Reserving slots for PAD and UNK
3. Filling the remaining $V - 848$ slots with the most frequent LIT tokens from the training corpus
4. Mapping all remaining LIT tokens to UNK

At 8K vocabulary, this produces a 5.6% UNK rate (rare words appearing fewer than 10 times). At 32K vocabulary, the UNK rate drops to 1.6%.

---

5. Experimental Setup

All code, tokenizer source, training scripts, and the paper are publicly available at **https://github.com/emadjumaah/cst**.

5.1 Dataset

We use 99,963 English sentences from the `agentlans/high-quality-english-sentences` dataset (Hugging Face), sourced from Wikipedia and covering diverse domains including science, history, geography, technology, and biography. The dataset provides broad lexical coverage without domain bias.

5.2 Tokenizer Configurations

We compare four tokenizer configurations in two matched pairs:

Configuration	Tokenizer	Vocab size	Description
CST-8K	CST (capped)	8,000	846 semantic + 7,152 frequent LIT + PAD + UNK
SPM-8K	SentencePiece BPE	8,000	Standard BPE trained on same corpus
CST-32K	CST (capped)	32,000	846 semantic + 31,152 LIT + PAD + UNK
SPM-32K	SentencePiece BPE	32,000	Standard BPE trained on same corpus

SentencePiece models are trained on the same 99,963 sentences using default BPE settings. This ensures both tokenizers see identical training data.

5.3 Model Architecture

All models use the GPT-2 architecture (Radford et al., 2019) as implemented in Hugging Face Transformers:

Hyperparameter	Value
Embedding dimension	256
Layers	6
Attention heads	4
Max sequence length	128
Learning rate	3×10^{-4}
Batch size	32
Optimizer	AdamW (weight decay 0.01)
Scheduler	Cosine annealing
Epochs	3
Validation split	10%

Within each vocabulary-matched pair, both models have **identical parameter counts**: 6.8M at 8K vocabulary (2.0M embedding + 4.7M transformer) and 13.0M at 32K vocabulary (8.2M embedding + 4.7M transformer). The only difference between paired models is the tokenization strategy.

5.4 Evaluation Metric

We use **bits-per-character (BPC)** as the primary evaluation metric:

$$\mathrm{BPC} = \frac{\sum_i \mathrm{NLL}_i}{C \cdot \ln 2}$$

where NLL_i is the per-token negative log-likelihood summed over all validation tokens, and C is the total number of characters in the original validation text.

BPC normalizes for differences in token sequence length and vocabulary size. Unlike perplexity, which operates in token space and is incomparable across tokenizers with different vocabularies, BPC measures how many bits the model requires to encode each character of the original text. Lower is better.

We also report per-token perplexity and average tokens per sentence for completeness, but emphasize that these metrics are not directly comparable across tokenizers.

---

6. Results

6.1 Main Results

Metric	CST-8K	SPM-8K	CST-32K	SPM-32K
Vocabulary	8,000	8,000	32,000	32,000
Parameters	6.8M	6.8M	13.0M	13.0M
Avg tokens/sentence	22.1	31.7	22.1	26.6
Val perplexity	113.0	163.9	172.8	302.5
Val BPC	**1.13**	**1.75**	**1.23**	**1.65**
BPC reduction		**35.5%**		**25.2%**

At 8K vocabulary, CST achieves a BPC of 1.13 compared to 1.75 for SentencePiece — a 35.5% reduction. At 32K vocabulary, CST achieves 1.23 versus 1.65 — a 25.2% reduction. Both comparisons use identical architectures with identical parameter counts.

6.2 Token Sequence Length

CST produces consistently shorter sequences: 22.1 tokens per sentence regardless of vocabulary cap, compared to 31.7 (SPM-8K) and 26.6 (SPM-32K). The stability of CST's sequence length across vocabulary sizes reflects the fact that semantic tokens are not affected by vocabulary capping — only rare LIT tokens are mapped to UNK, which does not change the token count.

The 30% shorter sequences have a direct computational benefit: CST-8K completes each training epoch in 102 seconds versus 159 seconds for SPM-8K on an NVIDIA T4 GPU, a 1.56× throughput improvement due to reduced sequence length in the self-attention computation.

6.3 Vocabulary Composition Effect

An unexpected finding is that CST-8K (1.13 BPC, 6.8M params) outperforms CST-32K (1.23 BPC, 13.0M params) despite having fewer parameters and a smaller vocabulary. We attribute this to vocabulary focus: at 8K, the vocabulary contains only high-frequency words alongside semantic tokens, reducing the sparsity of the embedding space. The 5.6% UNK rate at 8K effectively regularizes the model, forcing it to rely on semantic structure rather than memorizing rare surface forms.

This contrasts with SentencePiece, where the 32K vocabulary (1.65 BPC) outperforms the 8K vocabulary (1.75 BPC), following the expected pattern for subword tokenizers where larger vocabularies reduce segmentation granularity.

6.4 Training Dynamics

Metric	CST-8K	SPM-8K
Epoch 1 BPC	1.17	1.91
Epoch 2 BPC	1.14	1.78
Epoch 3 BPC	1.13	1.75
Epoch-over-epoch improvement	-0.02, -0.01	-0.13, -0.03

CST converges faster and starts from a lower initial loss. At epoch 1, CST-8K already achieves 1.17 BPC, a level that SPM-8K does not reach even after 3 epochs. SPM-8K shows larger epoch-over-epoch improvements, suggesting it may continue to narrow the gap with more training, but the initial advantage of structured tokenization is substantial.

---

7. Discussion

7.1 Why Does Semantic Tokenization Help?

We identify three mechanisms through which CST improves language modeling:

Sequence compression. CST represents each content word as a single token (ROOT or CMP), whereas BPE may split it into 2–4 subword fragments. Shorter sequences mean the model's self-attention operates over a more compact representation of the same information, reducing the distance between semantically related tokens.

Structured inductive bias. CST tokens explicitly encode semantic fields and morphological roles. The token `CMP:write:agent` signals that this word belongs to the *write* semantic cluster and functions as an *agent*. The model does not need to learn these relationships from raw co-occurrence statistics — they are provided directly in the input representation. This is the computational realization of the Arabic algebra: the model receives root × pattern as structured input rather than recovering it implicitly from data.

Field-level generalization. Words mapped to the same semantic field share a token prefix. "Writer," "writing," "written," and "wrote" all produce tokens beginning with `write`, differing only in role. This enables the model to generalize across morphological variants without needing to see each surface form in training. In Arabic, this generalization is automatic: any word derived from كتب shares root identity in the morphological representation. CST makes the equivalent generalization available to models trained on English.

7.2 Arabic as the Natural Next Step

The experiments in this paper are conducted on English. Arabic is, in a precise sense, the language for which CST requires the least additional work and where its advantages are largest.

In Arabic, the CST pipeline's morphological decomposition stage is replaced by a direct root-pattern extraction. Tools such as Farasa (Abdelali et al., 2016) and CAMeL Tools (Obeid et al., 2020) extract Arabic roots and patterns from surface forms with high accuracy. Where the English tokenizer detects the suffix *-er* in "writer" and looks up "write" in the semantic field dictionary, the Arabic tokenizer reads كاتب, identifies root كتب and pattern فاعل, and emits `CMP:write:agent` directly — the same token.

This means that an Arabic CST tokenizer produces a higher proportion of CMP tokens (lower LIT rate) than the English tokenizer, because Arabic morphological regularity makes more words decomposable. The 66.1% structured token rate achieved on English Wikipedia is a lower bound on what Arabic would achieve.

Furthermore, because the semantic tokens are identical across languages, a model pretrained on English CST tokens and then fine-tuned on Arabic CST tokens shares a common vocabulary for all semantic content. The cross-lingual transfer requires no alignment: `CMP:write:agent` means the same thing whether the training sentence came from English or Arabic.

7.3 Cross-Lingual Properties

A key architectural property of CST is that its semantic vocabulary (~846 tokens) is bounded and language-agnostic. The number of universal semantic fields is finite. As corpus size grows, the rate of new LIT tokens decreases sublinearly, since most content words eventually map to existing fields as the dictionary expands.

In contrast, BPE vocabularies scale linearly with linguistic diversity. A multilingual BPE tokenizer must allocate vocabulary slots to language-specific character sequences. CST's semantic fields are language-agnostic: *write* in English, *écrire* in French, كتب in Arabic, and 書く in Japanese all map to the same field. This has a direct consequence: two languages can share one model if they share the CST semantic token space — which they do by construction. An English sentence and its Arabic translation produce overlapping token sequences:

- English: `[CMP:write:agent] [CMP:send:past] [REL:to] [CMP:know:causer]`
- Arabic: `[CMP:write:agent] [CMP:send:past] [REL:to] [CMP:know:causer]`

The model learns in concept space. Languages are different surfaces over the same semantic structure.

Adding a new language to CST requires: (1) a morphological analyzer for that language, (2) language-specific entries in the relation and structure maps (~30 new entries), and (3) language-specific LIT tokens. The ~846 semantic tokens transfer unchanged. For morphologically rich languages — Arabic, Hebrew, Turkish, Finnish — where the root-pattern or agglutinative structure is explicit in word form, step (1) is well-supported by existing tools. The semantic algebra that Arabic revealed is the foundation; CST is the generalization.

7.4 UNK Rate as Implicit Regularization

The observation that CST-8K outperforms CST-32K is noteworthy. The 5.6% UNK rate at 8K acts as a form of input-level dropout, preventing the model from memorizing rare surface forms and forcing it to rely on the semantic structure of the remaining tokens. This mirrors findings in the dropout literature (Srivastava et al., 2014) where input-level noise can improve generalization by preventing co-adaptation. In CST's case, the "noise" is linguistically principled: only rare, low-frequency surface forms are collapsed to UNK, while the semantic structure remains intact. This suggests that aggressive vocabulary capping may be a useful technique for CST, particularly in low-resource settings where data sparsity makes embedding rare tokens counterproductive.

7.5 Limitations

Semantic field coverage. Our implementation covers approximately 2,400 lemma-to-field mappings. At 8K vocabulary with capping, 5.6% of token occurrences fall to UNK. Expanding the semantic dictionary would improve coverage, but even at current levels the model achieves strong results.

Dependency on NLP tooling. CST relies on compromise.js for lemmatization and NER. The quality of these components directly affects tokenization quality. Errors in lemmatization propagate to incorrect field resolution. Replacing the NLP backend with a more accurate tool would likely improve results.

Single-language evaluation. We evaluate on English only. While the semantic field framework is theoretically language-universal, and while the Arabic origin of the framework suggests strong applicability to Semitic languages, empirical validation on morphologically rich languages (Arabic, Turkish, Finnish) and typologically diverse languages (Mandarin, Japanese) is needed.

Small-scale experiments. Our experiments use 100K sentences and models up to 13M parameters. It remains an open question whether CST's advantages persist at the scale of modern language models. We observe consistent results across both vocabulary sizes (35.5% at 8K, 25.2% at 32K), suggesting the effect is not vocabulary-size dependent, though validation at larger scale remains future work.

Downstream task evaluation. We evaluate on language modeling (BPC) only. Evaluation on downstream tasks such as sentiment classification, question answering, and machine translation would provide a more complete picture of CST's utility.

---

8. Future Work

Arabic CST. The conceptual origin of this work is Arabic morphology. Implementing CST for Arabic using Farasa or CAMeL Tools for root-pattern extraction would validate the framework on the language that motivated it and is expected to yield higher structured token coverage than English. An empirical validation — training a single model on interleaved English and Arabic CST tokens — would test whether the shared semantic space enables genuine cross-lingual transfer without parallel data.

Multilingual CST. Beyond Arabic, CST is a natural fit for Hebrew (Semitic root system), Turkish (agglutinative morphology with explicit role suffixes), and Finnish (case-marked noun morphology). A multilingual semantic dictionary mapping lemmas from multiple languages to the same field set would enable a shared tokenizer with a compact, language-agnostic vocabulary.

Downstream task evaluation. Evaluation on classification (SST-2), question answering (SQuAD), natural language inference (MNLI), and machine translation benchmarks would establish whether BPC improvements translate to task-level gains.

Scaling experiments. Training larger models (100M+ parameters) on larger corpora (1M+ sentences) would clarify whether CST's advantages persist or diminish at scale.

Hybrid tokenization. Rather than falling back to LIT tokens for unresolved words, a hybrid approach could use subword segmentation as a fallback, combining CST's semantic precision with BPE's robustness for out-of-vocabulary words.

Learnable semantic fields. The current field dictionary is manually curated. An automatic method for inducing semantic fields from distributional data — for example, by clustering word embeddings and mapping clusters to field labels — could scale CST to open-vocabulary settings.

---

9. Conclusion

We have presented Contextual Semantic Tokenization, a linguistically-grounded tokenization method whose conceptual foundation is the algebraic structure of Arabic triconsonantal morphology. Arabic root × pattern composition — a system refined over fourteen centuries of linguistic use — encodes semantic field and morphological role directly in word structure. CST generalizes this principle to a universal framework, recovering the same structured information from English through lemmatization and affix detection, and encoding it as typed semantic tokens that any language model can train on.

In controlled experiments with identical model architectures and parameter counts, CST reduces bits-per-character by 35.5% at 8K vocabulary and 25.2% at 32K vocabulary compared to SentencePiece BPE. CST also produces 30% shorter token sequences, yielding faster training.

These results demonstrate that encoding linguistic structure directly into the tokenization layer provides a meaningful inductive bias for language modeling. The approach is simple to implement, requires no additional training beyond dictionary construction, and produces interpretable tokens. Its most natural extension is Arabic — the language whose morphological algebra made the principle visible — and from there to any language with recoverable morphological structure.

The Arabic root-pattern system was a formal algebra long before neural language models existed. CST is, in one sense, the observation that this algebra was always a tokenizer waiting to be used.

References

Abdelali, A., Darwish, K., Durrani, N., & Mubarak, H. (2016). Farasa: A fast and furious segmenter for Arabic. *Proceedings of NAACL-HLT: Demonstrations*.

Antoun, W., Baly, F., & Hajj, H. (2020). AraBERT: Transformer-based model for Arabic language understanding. *Proceedings of the LREC Workshop on Language Models and Their Applications*.

Ataman, D., & Federico, M. (2018). An evaluation of two vocabulary reduction methods for neural machine translation. *Proceedings of the 13th Conference of the Association for Machine Translation in the Americas*.

Baker, C. F., Fillmore, C. J., & Lowe, J. B. (1998). The Berkeley FrameNet project. *Proceedings of COLING-ACL*.

Beesley, K. R., & Karttunen, L. (2003). *Finite State Morphology*. CSLI Publications.

Bostrom, K., & Durrett, G. (2020). Byte pair encoding is suboptimal for language model pretraining. *Findings of EMNLP*.

Creutz, M., & Lagus, K. (2007). Unsupervised models for morpheme segmentation and morphology learning. *ACM Transactions on Speech and Language Processing*, 4(1).

Fillmore, C. J. (1982). Frame semantics. *Linguistics in the Morning Calm*.

Inoue, G., Alhafni, B., Baimukan, N., Bouamor, H., & Habash, N. (2021). The interplay of variant, size, and task type in Arabic pre-trained language models. *Proceedings of ARABICNLP*.

Kiraz, G. A. (2001). *Computational Nonlinear Morphology: With Emphasis on Semitic Languages*. Cambridge University Press.

Kudo, T. (2018). Subword regularization: Improving neural network translation models with multiple subword candidates. *Proceedings of ACL*.

Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *Proceedings of EMNLP: System Demonstrations*.

McCarthy, J. J. (1981). A prosodic theory of nonconcatenative morphology. *Linguistic Inquiry*, 12(3), 373–418.

Miller, G. A. (1995). WordNet: A lexical database for English. *Communications of the ACM*, 38(11).

Obeid, O., Zalmout, N., Khalifa, S., Taji, D., Oudah, M., Alhafni, B., Inoue, G., Eryani, F., Erdmann, A., & Habash, N. (2020). CAMeL Tools: An open source Python toolkit for Arabic natural language processing. *Proceedings of LREC*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Technical Report*.

Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., & Gurevych, I. (2021). How good is your tokenizer? On the monolingual performance of multilingual language models. *Proceedings of ACL*.

Schuster, M., & Nakajima, K. (2012). Japanese and Korean voice search. *IEEE International Conference on Acoustics, Speech, and Signal Processing*.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of ACL*.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

Trier, J. (1931). *Der deutsche Wortschatz im Sinnbezirk des Verstandes*. Heidelberg: Winter.

Appendix A: Tokenization Examples

Input: "The researchers discovered that rewriting the algorithm significantly improved computational efficiency."

Token	Type	Explanation
`STR:past`	STR	Past tense detected ("discovered")
`LIT:the`	LIT	Function word
`CMP:science:agent`	CMP	"researchers" → field *science*, role *agent*
`CMP:know:past`	CMP	"discovered" → field *know*, role *past*
`REL:that`	REL	Complementizer
`CMP:write:repeat`	CMP	"rewriting" → field *write*, role *repeat*
`LIT:the`	LIT	Function word
`ROOT:think`	ROOT	"algorithm" → field *think*
`CMP:quality:manner`	CMP	"significantly" → field *quality*, role *manner*
`CMP:fix:past`	CMP	"improved" → field *fix*, role *past*
`CMP:think:quality`	CMP	"computational" → field *think*, role *quality*
`CMP:work:state`	CMP	"efficiency" → field *work*, role *state*

BPE-8K segmentation:

```
_The _research ers _discover ed _that _re writ ing _the _algorithm _significant ly _improv ed _comput ati
```

(17 tokens)

CST: 12 tokens. **BPE-8K:** 17 tokens. CST captures semantic relationships that BPE fragments lose.

Cross-lingual example: The Arabic equivalent — "اكتشف الباحثون أن إعادة كتابة الخوارزمية حسّنت الكفاءة الحسابية بشكل ملحوظ" — produces a largely overlapping CST sequence, because Arabic morphology delivers the root-pattern decomposition directly:

Token	Type	Arabic source
`STR:past`	STR	Sentence-level past tense
`CMP:know:past`	CMP	اكتشف — root ك-ش-ف (know), pattern افتعل (past)
`CMP:science:agent`	CMP	الباحثون — root ب-ح-ث (science), pattern فاعل (agent)
`REL:that`	REL	أن
`CMP:write:repeat`	CMP	إعادة كتابة — root ك-ت-ب (write), إعادة (repeat)
`ROOT:think`	ROOT	الخوارزمية — field *think*
`CMP:fix:past`	CMP	حسّنت — root ح-س-ن (fix), pattern فعّل (past)
`CMP:work:state`	CMP	الكفاءة — field *work*, role *state*
`CMP:think:quality`	CMP	الحسابية — field *think*, role *quality*

The semantic tokens are identical. Only language-specific function words differ. This is the cross-lingual property of CST made concrete: Arabic morphology directly produces the same token representation that English requires a processing pipeline to recover.

Appendix B: Semantic Field Inventory (Excerpt)

Field	Representative lemmas	Semantic domain
`think`	consider, reason, analyze, compute, calculate	Cognition
`move`	go, travel, arrive, depart, migrate, flow	Motion
`create`	make, build, produce, generate, design, invent	Creation
`know`	learn, understand, study, teach, educate	Knowledge
`write`	record, document, note, publish, inscribe	Written expression
`send`	transmit, dispatch, deliver, forward, mail	Transfer
`see`	observe, watch, notice, perceive, detect	Perception
`speak`	say, tell, announce, declare, communicate	Oral expression
`feel`	sense, experience, suffer, enjoy	Emotion
`govern`	rule, regulate, administer, legislate	Authority
`trade`	buy, sell, exchange, import, export	Commerce
`fight`	attack, defend, resist, conquer, invade	Conflict
`dwell`	live, reside, inhabit, settle, shelter	Habitation
`health`	heal, cure, treat, diagnose, infect	Medicine
`nature`	grow, bloom, evolve, decay, erode	Natural process

These fields originate from the Arabic Algebra Engine's root dictionary, which organized 820+ Arabic roots into semantic domains. The fields were subsequently found to be universal — the same semantic domains appear in English, and the same Arabic roots that anchor each domain map to the same English lemma clusters.

Appendix C: Experimental Reproducibility

All code, data pipelines, and training scripts are available in the project repository. The experiment can be reproduced as follows:

1. **Data preparation:** 99,963 sentences from `agentlans/high-quality-english-sentences` (Hugging Face), downloaded via Parquet format.
2. **CST tokenization:** TypeScript pipeline using compromise.js v14.14.3 for lemmatization and NER.
3. **SentencePiece training:** `sentencepiece` Python library v0.2.1, BPE mode, trained on the same 99,963 sentences.
4. **Vocabulary capping:** CST vocabularies capped to 8K and 32K by retaining all semantic tokens and top-frequency LIT tokens.
5. **Model training:** GPT-2 via Hugging Face Transformers, trained on Google Colab with NVIDIA T4 GPU.
6. **Evaluation:** BPC computed as total validation NLL (nats) divided by total validation characters divided by ln(2).

The full codebase — tokenizer (TypeScript), training scripts (Python), vocabulary capping utilities, and this paper — is available at **https://github.com/emadjumaah/cst**. The semantic field dictionary and Arabic Algebra Engine that motivated this work are available at https://emadjumaah.github.io/aae/.